PDF-версия исходной лекции iSMART из сохранённого HTML. Содержание не заменено пересказом. Служебные элементы интерфейса исключены.

Учебные данные синтетические. Не используйте реальные персональные и конфиденциальные сведения.

1. Паспорт самостоятельной работы

Работа направлена не только на описание прототипа, но и на проверку согласованности всех частей документа. Требования должны объяснять, что обязана делать система, архитектура — как это реализуется, а тесты — какими наблюдаемыми результатами подтверждается выполнение требований.

Содержание

- 3. Семь проверяемых шагов
- 4. Рекомендуемое распределение времени
- 6. Что необходимо сдать
- 7. Дополнительная P2P-проверка
- 8. Итог работы

Параметр	Содержание
Цель	Подготовить воспроизводимое техническое описание, связывающее требования, архитектуру, данные, правила обработки, тесты и порядок использования.
Продолжительность	45 минут.
Форма	Индивидуальная самостоятельная работа.
Итог	Семь отдельных результатов по проекту CampusSupport Assistant 2.2.
Оценивание	35 баллов: по 5 баллов за каждый проверяемый шаг.
Обязательная часть	Автоматическая проверка структуры технического описания.

Важно для сдачи
Единый проект является контекстом, но не одним заданием. Шаги 1–7 оформляются отдельно; каждый шаг имеет собственный результат и оценивается независимо.

> **Обратите внимание**
>
> Связность документа не означает объединение всех ответов в один текст. Каждый результат должен быть самостоятельным, но при этом не противоречить результатам других шагов. Например, тест не может проверять поле или правило, которое отсутствует в спецификации.

3. Семь проверяемых шагов

Шаги образуют последовательность доказательств. Сначала фиксируется ожидаемое поведение, затем описываются данные и механизм обработки, после чего результат проверяется тестами и итоговым аудитом.

№	Шаг	Что нужно сдать
1	Качество требований	Исправленные формулировки четырёх неоднозначных требований.
2	Спецификация и границы	Три функциональных требования, одно нефункциональное требование и границы системы.
3	Архитектура и паспорт данных	Схема компонентов, паспорт набора и словарь пяти полей.
4	Подход и правила обработки	Обоснование **rule-based подхода**, альтернатива и последовательность работы решения.
5	Тестирование и доказательства	Журнал шести сценариев, расчёт показателей и вывод о качестве.
6	Инструкция, ограничения и улучшения	Воспроизводимая инструкция, два ограничения, два риска и два улучшения.

№	Шаг	Что нужно сдать
7	Автоматическая проверка и итоговый аудит	Функция `validate_technical_spec`, результаты тестов и вывод о готовности.

4. Рекомендуемое распределение времени

Распределение является ориентиром. Если отдельный шаг выполнен раньше, оставшееся время целесообразно использовать для проверки связей между требованиями, тестами и выводами, а не для добавления непроверяемых описаний.

Этап	Время
Ознакомление с контекстом	3 минуты
Шаг 1	5 минут
Шаг 2	6 минут
Шаг 3	6 минут
Шаг 4	5 минут
Шаг 5	8 минут
Шаг 6	6 минут
Шаг 7	4 минуты
Самопроверка и сдача	2 минуты

6. Что необходимо сдать

Перед отправкой проверьте, что каждый результат можно открыть и оценить отдельно, а названия полей, версий, категорий и действий записаны одинаково во всех разделах.

- шаг 1 — четыре исправленных требования;
- шаг 2 — спецификацию и границы решения;
- шаг 3 — паспорт данных, словарь полей и архитектуру;
- шаг 4 — описание подхода и последовательность обработки;
- шаг 5 — журнал тестов, показатели и вывод;

- шаг 6 — инструкцию, ограничения, риски и улучшения;
- шаг 7 — функцию `validate_technical_spec` и финальный аудит;

7. Дополнительная P2P-проверка

Необязательная активность
После сдачи партнёр может выбрать одно требование, один раздел данных и один вывод по тестированию. Он не редактирует документ, а оставляет одну конкретную проверяемую рекомендацию.

Полезная рекомендация указывает место, проблему и способ проверки исправления. Формулировка «улучшить требования» слишком общая; рекомендация должна показывать, какое именно условие, значение или критерий отсутствует.

Критерий	Да / частично / нет	Комментарий
Требования содержат условие и ожидаемый результат.		
Критерии приёмки измеримы.		
Описание данных содержит источник, поля и ограничения.		
Подход назван корректно и обоснован.		
Инструкция воспроизводима.		

8. Итог работы

Вы подготовили семь самостоятельных результатов технического описания: сделали требования проверяемыми, описали границы и архитектуру, оформили паспорт данных, обосновали подход, зафиксировали результаты тестов, подготовили инструкцию и выполнили автоматическую проверку структуры. Такой формат позволяет оценить не один общий кейс, а каждый обязательный результат отдельно.
Готовое техническое описание должно отвечать на три итоговых вопроса: что именно обязано делать решение, как воспроизвести его работу и какими доказательствами подтверждён результат. Если для каждого утверждения можно найти требование, компонент, тест или ограничение, документ является связным и пригодным для проверки.

Контекст и базовая архитектура CampusSupport Assistant 2.2

Прототип получает обращение пользователя и возвращает категорию, приоритет и способ обработки. Он демонстрирует прозрачную маршрутизацию по словарям и правилам и не заменяет промышленную систему Service Desk.
Прозрачность означает, что для каждого результата можно проследить цепочку решений: какие входные значения были приняты, какое правило сработало и почему было выбрано конкретное действие. Это особенно важно при ручной трассировке и разборе неуспешных тестов.

Компонент	Сведения
Пользователи	`student`, `teacher`, `administrator`
Каналы	`portal`, `email`, `phone`
Вход	`text`, `role`, `channel`, `confidence`
Категории	`access`, `network`, `equipment`, `software`, `other`
Приоритеты	`P1` — критический; `P2` — повышенный; `P3` — обычный
Действия	`auto_route`, `operator_review`, `input_error`
Порог уверенности	`confidence < 0.65` → `operator_review`
Критические фразы	«сервер недоступен», «утечка данных», «не работает сеть во всём корпусе»

Значение на самой границе требует точного чтения условия: правило задано строгим неравенством. Следовательно, значение `0.64` ниже порога, а значение `0.65` под условие `confidence < 0.65` не попадает.

Логическая архитектура
Вход → валидация типов и допустимых значений → нормализация текста → определение категории → определение приоритета → выбор действия → формирование результата → журнал тестирования.

Порядок компонентов существенен. Например, недопустимую роль необходимо обнаружить на этапе валидации до автоматической маршрутизации, иначе система может сформировать внешне корректный результат на некорректном входе.